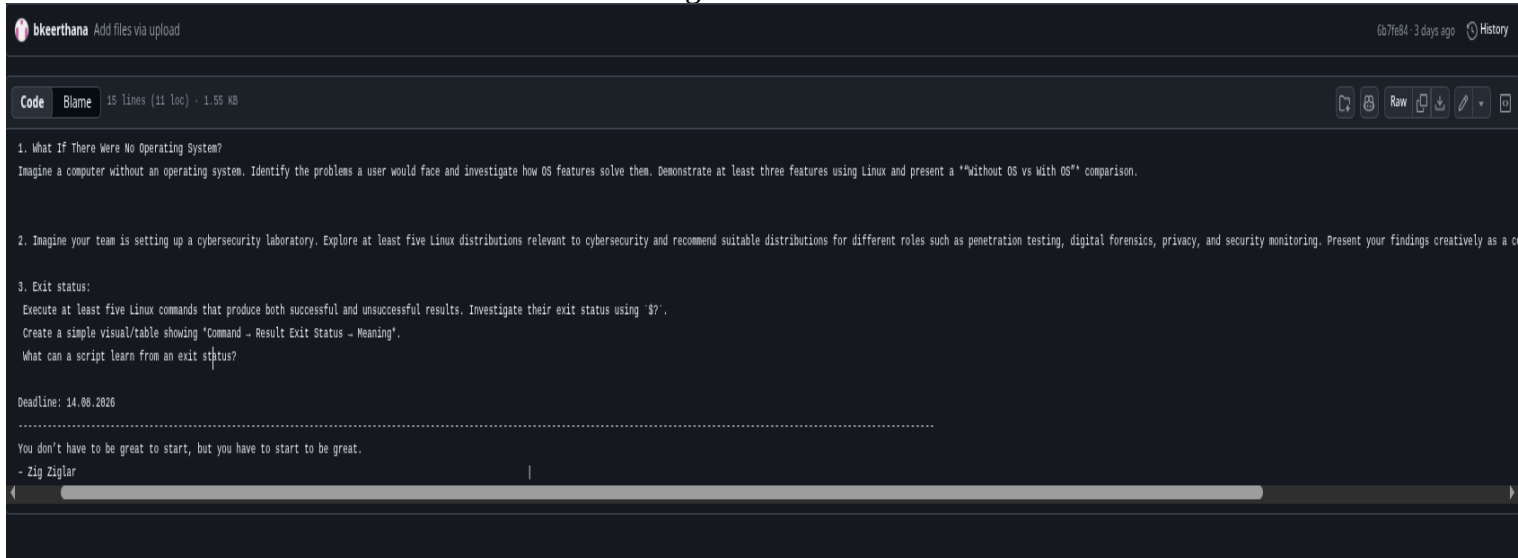


R.K Ayushman Singh
Roll No:261100690014
Date:10/08/26

LOS Assignment-1



1. What if there was no OS?

- If there was no operating system, using a computer would basically be impossible for a user. The OS is what sits between user and the hardware and does all the hard work in the background.

Problem that would happen:

- It can't run more than one program at same time, you would have to finish one then load next manually.[1]
- No concept of files/folders, you would need to know exact memory location to save and access anything.[2]
- No security, any program can access any file, no permissions are required.[3]
- There is no way to manage multiple programs using CPU/RAM at same time, they would clash.[4]
- Every device (Keyboard, printer, mouse etc) would need separate custom code to work, since no drivers.[5]

How OS fixes them:

- [1] It fixes this using a process scheduling, gives each program small turns of CPU so fast it looks like multitasking.
- [2] It gives file system so we just do ls, cd instead.
- [3] It handles this with permissions, decides who can read/write/execute.
- [4] It gives each one separate memory space.
- [5] It uses drivers so programs don't need to know exact hardware details.

Demo using linux(3 features):

- **File system:**

Got some folder names instead of raw disk addresses. This is the file system features solving the problem.

```
anonymous@anonymous-supreme in ~ via v3.14.6
$ ls
-rwxr-xr-x - anonymous 30 Jul 10:14 .anaconda
-rwxr-xr-x - anonymous 11 Aug 16:10 .cache
-rwxr-xr-x - anonymous 8 Sep 2025 .conda
-rwxr-xr-x - anonymous 11 Aug 16:13 .config
-rwx----- - anonymous 2 Aug 11:42 .copilot
-rwxr-xr-x - anonymous 2 Sep 2025 .dotnet
-rwx----- - anonymous 5 Sep 2025 .firedragon
-rwxr-xr-x - anonymous 8 Mar 2025 .icons
-rwxr-xr-x - anonymous 28 Sep 2025 .ipynb_checkpoints
-rwxr-xr-x - anonymous 5 Sep 2025 .ipython
-rwx----- - anonymous 7 Sep 2025 .john
-rwxr-xr-x - anonymous 11 Sep 2025 .jupyter
-rwxr-xr-x - anonymous 11 Sep 2025 .local
-rwxr-xr-x - anonymous 8 Sep 2025 .npm
```

- **Process management:**

As we can see here the multiple processes running together ,each with own PID,done by the OS.so this is the process scheduling solving the problem one program at a time.

```
anonymous@anonymous-supreme in ~ via v3.14.6
➤ ps aux | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.2 21708 16224 ?        Ss   15:18   0:01 /usr/lib/systemd/systemd -
ched-root --system --deserialize=59
root         2  0.0  0.0      0     0 ?        S    15:18   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    15:18   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   15:18   0:00 [kworker/R-rcu_gp]
```

- **Permission:**

It got blocked from writing into system file,exit code 1 confirms it failed

```
anonymous@anonymous-supreme in ~ via v3.14.6 as
➤ touch /etc/shadow-test
touch: cannot touch '/etc/shadow-test': Permission denied

anonymous@anonymous-supreme in ~ via v3.14.6 as
[*] ➤ echo $status
1
```

Without OS	With OS
One program at a time	Multitasking
Manual disk address	Files and folders
No Security	Permissions
Program clash in memory	Separate memory per process
Custom code per device	Drivers handle

2.Linux distros for cybersecurity lab.

- Made a small table to showcase the 6 distros:

Distro	Used for	Why
Kali linux	Pentesting	It comes with tools like nmap,metasploit ..etc already installed
Parrot OS	Pentesting +privacy	It is similar to kali but lighter,has privacy tools too.
CAINE	Digital forensics	It is made for evidence collection, keeps chain of custody safe.
Security Onion	Monitoring	It is used for network monitoring and intrusion detection
BlackArch	Advanced Pentesting	It is arch based,And has even more tools than kali
Tails	Privacy	It runs from usb,routes evrythingh through Tor,and it doesn't save anything after shutdown

If i wanted to recommend this to anyone it would be like :

- Pentesting=Kali or Parrot
- Forensics=CAINE
- privacy =Tails
- Monitoring=Security Onion

3. Exit Status

```

anonymous@anonymous-supreme in ~ via v3.14.6 as
#) ls /home
drwx----- anonymous 11 Aug 15:21 anonymous
anonymous@anonymous-supreme in ~ via v3.14.6
#) echo $status
0

anonymous@anonymous-supreme in ~ via v3.14.6
#) ls /hume
"/hume": No such file or directory (os error 2)
anonymous@anonymous-supreme in ~ via v3.14.6
[*] # echo $status
2

anonymous@anonymous-supreme in ~ via v3.14.6
#) echo hello | grep hello
hello
anonymous@anonymous-supreme in ~ via v3.14.6
#) echo $status
0

anonymous@anonymous-supreme in ~ via v3.14.6
#) echo hello | grep h1llo
[[!][*]] => *ERROR # echo $status
1

anonymous@anonymous-supreme in ~ via v3.14.6
#) mkdir testdir
anonymous@anonymous-supreme in ~ via v3.14.6
#) echo $status
0

anonymous@anonymous-supreme in ~ via v3.14.6
#) mkdir testdir
mkdir: cannot create directory 'testdir': File exists
anonymous@anonymous-supreme in ~ via v3.14.6
[*] # echo $status
1

anonymous@anonymous-supreme in ~ via v3.14.6
#) touch newfile.txt
anonymous@anonymous-supreme in ~ via v3.14.6
#) echo $status
0

anonymous@anonymous-supreme in ~ via v3.14.6
#) touch /etc/shadow-test
touch: cannot touch '/etc/shadow-test': Permission denied
anonymous@anonymous-supreme in ~ via v3.14.6
[*] # echo $status
1

anonymous@anonymous-supreme in ~ via v3.14.6
#) cd /home
anonymous@anonymous-supreme in /home
#) echo $status
0

anonymous@anonymous-supreme in /home
#) cd /homa
cd: The directory '/homa' does not exist
anonymous@anonymous-supreme in /home
[*] # echo $status
1

```

Command->Result->Exit Status->Meaning:

Command	Case	Result	Exit Status	Meaning
ls/home	success	Listed folder contents	0	success
ls/hume	fail	No such directory	2	Failure, folder doesn't exist
echo hello grep hello	success	Match found	0	Success
echo hello grep h1llo	fail	No match found	1	Failure, nothing matched
mkdir testdir	success	Folder created	0	success
mkdir testdir	fail	Folder already exists	1	Failure, can't create duplicate
touch newfile.txt	success	File created	0	success
touch /etc/shadow-test	fail	Permission denied	1	Failure, no write access
cd /home	success	Directory changed	0	success
cd /homa	fail	No such directory	1	Failure, folder doesn't exist

What can a script learn from exit status:

As per my knowledge scripts checks the exit code instead of reading actual text output. 0 means step worked so it moves on, non-zero means something failed so script can print either error, retry or stop completely. It is useful in automation like backup or deployment scripts.